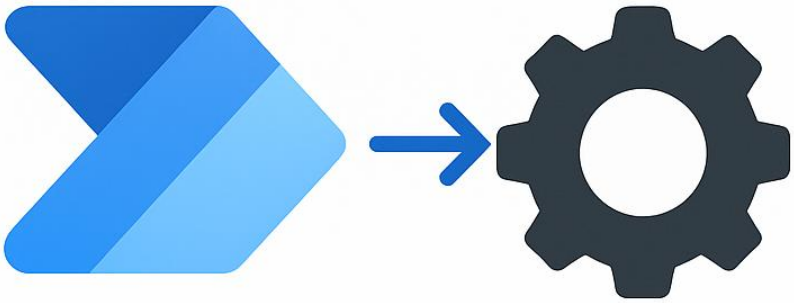
Dynamics 365 Workflows and Power Automate Flows

A Complete Practical and Technical Guide



Usama Sarfraz

Microsoft Certified:
Dynamics CRM Developer/Consultant

1. Introduction

Workflows and Power Automate flows are essential automation tools inside Microsoft Dynamics 365. They automate repetitive processes, enforce business rules, improve data consistency, and reduce manual effort for end users.

Classic Dynamics 365 Workflows run directly inside the Dataverse workflow engine. They are very good at internal CRM logic: updating records, enforcing validations, setting statuses, sending notifications, and coordinating business rules for a single business application.

Power Automate is the modern cloud automation platform from Microsoft. It connects Dynamics 365 with Microsoft 365 (Outlook, SharePoint, Teams), Azure, and hundreds of external services through connectors. It is the first choice for integrations, approvals, file processing, cross-system data sync, and "low-code" automation that business users can maintain.

Custom Workflow Activities (CWA) extend workflows with C# code. They give you plugin-style power inside the workflow designer and let you perform logic that is not possible with standard workflow actions.

This guide explains how these three automation options fit together, how to design them correctly, what errors you should expect, and which best practices help you build solutions that are stable in production and easy to maintain.

2. Dynamics 365 Workflows

2.1 What are Workflows?

Workflows are server-side processes that run in the Dataverse workflow engine. They are:

- **Event-driven:** they respond to record events such as create, update, delete, assign, or status change.
- **Server-side:** they run regardless of which client the user is using (model-driven app, Outlook, API, portal, etc.).
- **Configurable:** you design the logic using the workflow designer, without writing C# for most scenarios.

Workflows are ideal when the logic:

- Should always run inside CRM
- Must be close to the data
- Needs to run in real time as part of saving a record, or in the background shortly after

2.2 Creating a Workflow (Step by Step)

1. Go to **Settings → Processes → New**.
2. Enter a **Process Name** (for example, *Case – Auto Route by Severity*).
3. Select the **Category** as **Workflow**.

4. Select the **Entity** you want to run on (for example, *Case*).
5. Choose **Run this workflow in the background** = Yes/No depending on whether you need real-time or background execution.
6. Click **OK** to open the process designer.
7. In **Start when** options, choose the events (Create, Update, Assign, etc.).
8. If you choose Update, select specific fields under **Record fields change** to avoid unnecessary runs.
9. Add workflow steps: conditions, Create/Update Record, Assign, Send Email, Wait Conditions, and so on.
10. Save, **Activate** the workflow, and test using realistic data.

2.3 Workflow Triggers (Events)

Workflows can start on:

- **Record is created**
  Useful for initialization logic, default values, or creating related records.
- **Record is updated**
  Combine this with **specific field selection** so the workflow only runs when relevant data changes. This is critical to avoid recursion loops.
- **Record is deleted**
  Used for clean-up, logging, or cascading updates.
- **Record is assigned**
  When ownership changes, you can send notifications, set follow-up dates, or update routing fields.
- **Record status changes**
  Used for lifecycle automation (case resolved, opportunity won/lost, custom status fields).
- **On-demand**
  Users can run the workflow manually from the ribbon on selected records.
- **Child workflow**
  Another workflow can call this one, which is ideal for reusing logic instead of copying steps.

2.4 Workflow Actions

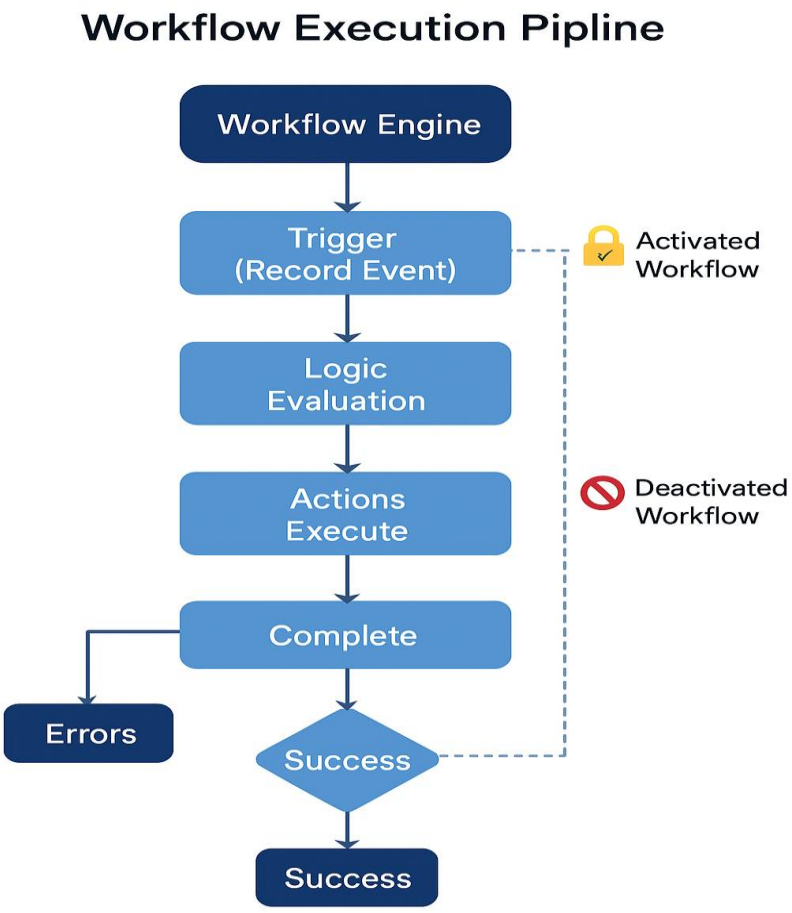
Some of the most common actions are:

- **Create Record** – Create related cases, tasks, emails, notes, or custom entities.
- **Update Record** – Set fields such as status, owner, score, flags, or calculated fields.
- **Assign Record** – Change owner to a user or team.

- **Send Email** – Send notifications using templates or dynamic data.
- **Change Status** – Move a record through its lifecycle (e.g., Active → Resolved).
- **Start Child Workflow** – Execute reusable logic defined in a child workflow.
- **Stop Workflow** – Stop with **Succeeded** or **Cancelled** (with an error message).
- **Wait Condition** – Pause the workflow until a date/time, a field value, or a related condition is met.

2.5 Workflow Execution Pipeline

The workflow execution pipeline begins when a configured event occurs on a record. The workflow engine evaluates the trigger, checks conditions, executes steps, and then either completes successfully or throws an error, which appears in System Jobs. Real-time workflows run during the transaction and can prevent the record from being saved. Background workflows run after the transaction completes and never block the user.



2.6 Common Workflow Errors and How to Fix Them

1. Infinite/Unwanted Recursion

- **Symptom:** Workflow keeps re-running on the same record; System Jobs show many identical executions.
- **Cause:** Update step changes a field that is part of the workflow's "start on update" trigger.
- **Fix:**
    - Use explicit field change filters.
    - Add a flag field (e.g. *Workflow Processed*) and check it before updating.

2. "The workflow could not be run because the user does not have sufficient privileges"

- **Cause:** The workflow owner does not have permission on some records or entities the workflow is trying to access.
- **Fix:**
    - Change the workflow owner to a service account or System Administrator.
    - Ensure proper security roles on related entities.

3. Workflows stuck in "Waiting"

- **Cause:** Wait condition is never met or depends on missing data.
- **Fix:**
    - Avoid deep chains of Wait steps.
    - Use shorter waits combined with scheduled Power Automate flows when needed.
    - Review the Wait condition logic carefully.

4. SQL Timeout / "The operation has timed out"

- **Cause:** Large data operations, multiple nested workflows, or heavy plugins combined with workflows.
- **Fix:**
    - Simplify the workflow and move heavy work to background workflows or Power Automate.
    - Review plugins that might run on the same message.

2.7 When to Use Workflows vs Other Options

Use **Workflows** when:

- Logic must run **inside CRM** and close to the data.

- You need **real-time validation** before saving a record.
- The logic is mostly **CRUD on a single business application**.

Use **Power Automate** when:

- You need **integrations** with Outlook, SharePoint, Teams, other systems.
- You require approvals and multi-step, cross-system orchestration.

Use **Custom Workflow Activities or Plugins** when:

- You need **complex, reusable C# logic** beyond workflow actions.

3. Custom Workflow Activities (CWA)

3.1 What is a CWA?

A Custom Workflow Activity is a C# class that inherits from Code Activity and can be used as a step in a workflow. CWAs run inside the Dataverse sandbox, the same way plugins do, and have access to:

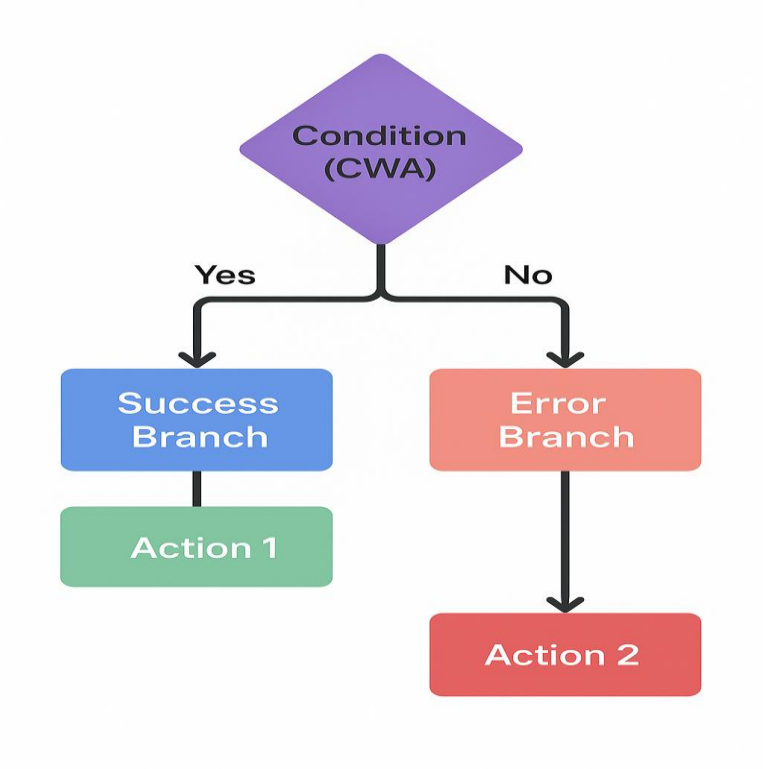
- IOrganizationService – to query and update data.
- ITracingService – to log information for debugging.
- IWorkflowContext – to understand which record triggered the workflow.

Use a CWA when standard workflow actions cannot implement your logic, for example:

- Complex calculations
- Lookup logic spanning several related entities
- Custom text manipulation or parsing
- Shared logic that will be reused in many workflows

3.2 CWA Architecture

The workflow calls the CWA as a step. If the CWA completes without throwing an exception, the workflow continues along the success path. If it throws an exception, the workflow goes down the error branch and appears as failed in System Jobs.



3.3 Developing a CWA – High Level Steps

1. Create a **Class Library** project (typically .NET Framework 4.6.2 for on-prem / plugin-style development).
2. Add references to Microsoft.Xrm.Sdk and System.Activities.
3. Create a class that inherits from CodeActivity.
4. Define **Input** and **Output** arguments as properties with attributes like [Input("Subject")].
5. Implement protected override void Execute(CodeActivityContext context).
6. Use context.GetExtension<IOrganizationServiceFactory>() to create IOrganizationService.
7. Use context.GetExtension<ITracingService>() for logging.
8. Build the assembly and register it using the **Plugin Registration Tool** as a workflow activity.
9. Open the workflow designer; the CWA appears in the list of available actions.
10. Configure the inputs in the workflow and test.

3.4 CWA Code Example: Create Follow-Up Task

```
using System;
using System.Activities;
using Microsoft.Xrm.Sdk;
using Microsoft.Xrm.Sdk.Workflow;

namespace Contoso.Workflows
{
    public class CreateFollowUpTask : CodeActivity
    {
        [Input("Subject")]
        public InArgument<string> Subject { get; set; }

        protected override void Execute(CodeActivityContext context)
        {
            var tracing = context.GetExtension<ITracingService>();
            tracing.Trace("CreateFollowUpTask CWA execution started.");

            try
            {
                var factory = context.GetExtension<IOrganizationServiceFactory>();
                var service = factory.CreateOrganizationService(null);

                // Get the primary entity from the workflow context
                var wfContext = context.GetExtension<IWorkflowContext>();
                if (wfContext == null)
                {
                    tracing.Trace("Workflow context is null; exiting.");
                    return;
                }

                // Create a new Task related to the primary record
                var task = new Entity("task");
                task["subject"] = Subject.Get(context) ?? "Follow-up Task";
                task["description"] = "Task created via Custom Workflow Activity.";
                task["regardingobjectid"] = new EntityReference(
                    wfContext.PrimaryEntityName,
                    wfContext.PrimaryEntityId);

                service.Create(task);
                tracing.Trace("Task created successfully.");
            }
            catch (Exception ex)
            {
                tracing.Trace("ERROR in CreateFollowUpTask: {0}", ex.ToString());
                throw new InvalidPluginExecutionException("CreateFollowUpTask failed.", ex);
            }
        }
    }
}
```

3.5 CWA Best Practices

- Always use ITracingService for start, key steps, and errors.
- Validate all inputs and handle nulls gracefully.
- Wrap logic in try/catch and throw InvalidPluginExecutionException with a clear message.
- Keep each CWA focused on a single responsibility.
- Avoid heavy loops and long-running operations (remember it runs in the sandbox).
- Prefer small, reusable CWAs rather than one big “God” activity.

4. Power Automate Cloud Flows

4.1 What is Power Automate?

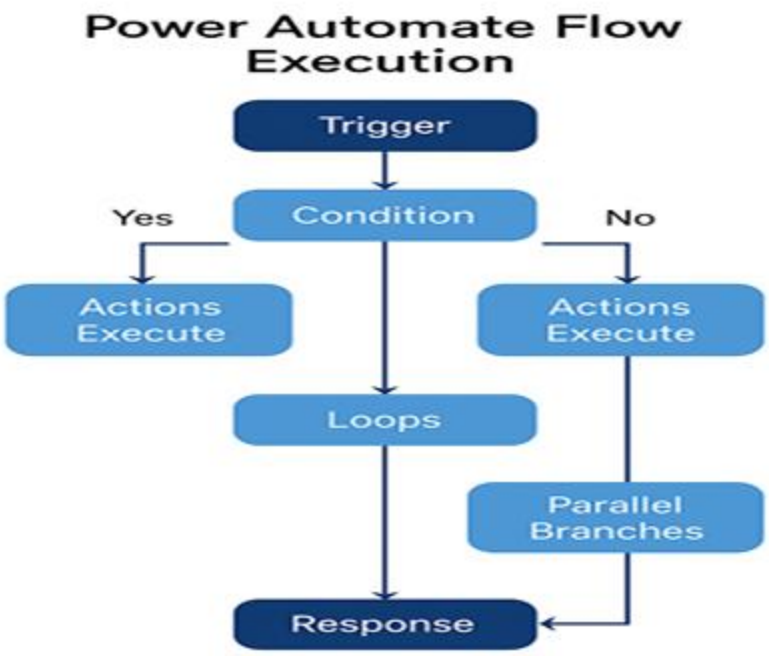
Power Automate is the cloud automation engine that connects Dynamics 365 to other services. Flows are built in the browser using a visual designer and use connectors to talk to Dataverse, SharePoint, Outlook, Teams, HTTP endpoints, SQL, and many more systems.

Types of flows you are using in this guide:

- **Automated flows** – Start from a trigger (e.g., Dataverse row added).
- **Instant flows** – Started manually (button in app, Power Apps, or via API).
- **Scheduled flows** – Run on a recurrence (every X minutes, hours, days).

4.2 Flow Execution Architecture

Every flow starts with a trigger. After the trigger fires, conditions and branching logic determine which actions run. Actions can include Dataverse operations, HTTP calls, loops, approvals, parallel branches, and file processing. The flow continues until it reaches a logical end or a Terminate action. Error handling is implemented using Scopes, Run After conditions, retry policies, and custom status fields.



4.3 Trigger Categories

Dataverse Triggers

- When a row is added, modified, or deleted in a Dataverse table.
- Support change tracking and filtering attributes.
- Ideal for replacing many classic workflows.

SharePoint Triggers

- When an item or file is created/modified.
- Used for document management and list automation, such as when SharePoint is the main document store for a CRM entity.

HTTP Trigger

- Exposes a URL that external systems can call to start the flow.
- Supports JSON payloads and can respond with a custom body.
- Good for integrating external systems without building a full API.

Outlook Triggers

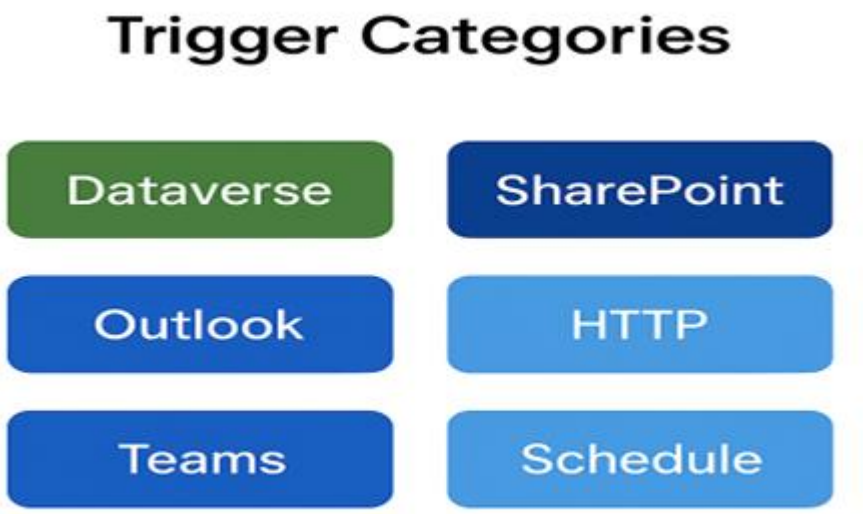
- When a new email arrives, when a flag is set, or when a calendar event is created.
- Useful for case creation from emails, tracking approvals, and notifications.

Teams Triggers

- When a message is posted in a channel, when a keyword is used, or when a new team is created.
- Good for integrating CRM updates into Teams collaboration.

Schedule Trigger

- Recurrence trigger for periodic jobs such as daily sync, cleanup, or reporting.



4.4 Flow Capabilities and Patterns

- **Conditions & Switch** – Control logic branches.
- **Loops (Apply to each, Do until)** – Process multiple records.
- **Parallel branches** – Run independent actions at the same time.
- **Scopes** – Group actions and handle success/failure as a block.
- **Child flows** – Reusable flows called from other flows.
- **Approvals** – Built-in approval templates that integrate with Outlook and Teams.

4.5 Common Flow Errors and Troubleshooting

1. Throttling / 429 Too Many Requests

- Cause: Hitting API limits on Dataverse or other connectors.

- Fix:
    - Reduce frequency of triggers.
    - Use pagination and batching.
    - Enable concurrency control and lower the degree of parallelism.

2. Flow keeps triggering itself (loop)

- Cause: Flow updates the same row that triggers it.
- Fix:
    - Use trigger conditions to ignore updates done by the flow.
    - Use a “Processed” flag or version field.

3. “BadGateway” / connector intermittent failures

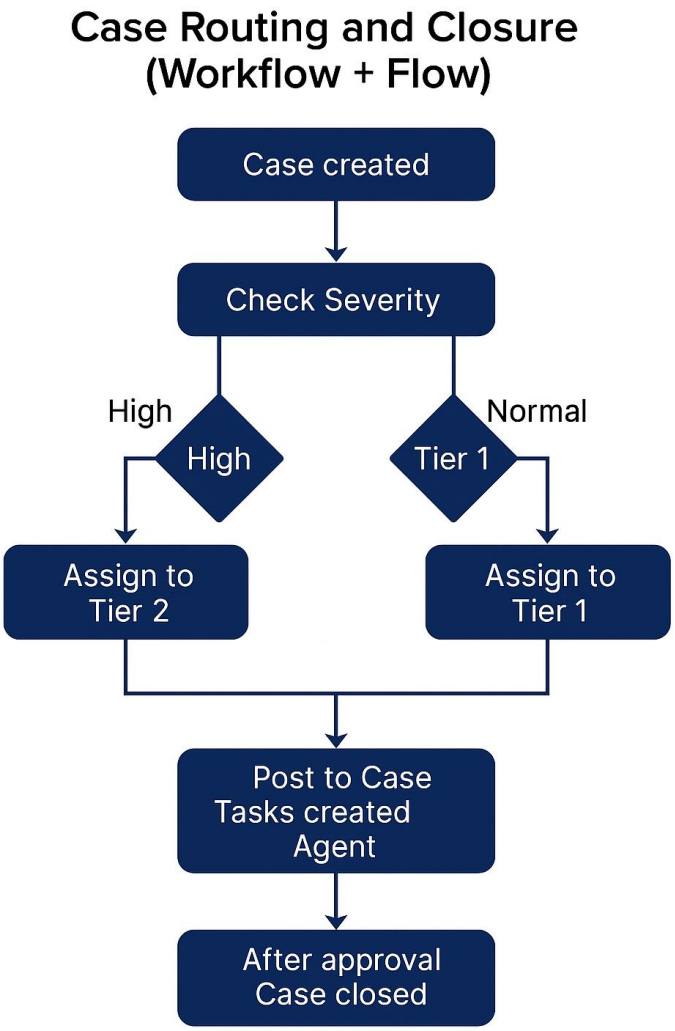
- Fix:
    - Turn on retry policy or configure custom retry in actions.
    - Use Scopes with **Run After** conditions to implement try/catch style logic.

4. Permission / authentication errors

- Fix:
    - Check the connection owners and service accounts.
    - Make sure the connector uses an account that has sufficient privileges in Dataverse / SharePoint, etc.

5. Real-World Scenarios

5.1 Case Routing and Closure (Workflow + Flow)



Process Description:

1. **Case is created** (through portal, email-to-case, or directly by user).
2. A **real-time workflow** or **Dataverse-triggered flow** checks the **Severity** field.
3. If severity is High/Critical:
    - Assign the case to **Tier 2** queue or team.
    - Create a Task with 2-hour SLA.
    - Send a notification to on-call team in Teams.

4. If severity is Normal:
    - Assign the case to **Tier 1**.
    - Create a Task with standard SLA.
5. When the agent resolves the case:
    - A workflow validates that mandatory fields (Resolution, Category, Root Cause) are set.
    - If validation passes, the case status is changed to **Resolved**.
    - If validation fails, workflow cancels the status change and shows an error.
6. A scheduled Power Automate flow closes resolved cases automatically after X days and sends a satisfaction survey.

5.2 Lead Qualification Flow

Outline:

- Lead is created from website or import.
- Automated flow scores the lead based on source, interest, geography.
- If score is above threshold:
    - Create Account + Contact.
    - Create an Opportunity.
    - Assign to sales owner based on territory.
- If score is below threshold:
    - Add to a nurture marketing list.
    - Send a follow-up email sequence.

5.3 Invoice Validation and Approval

Outline:

- Invoice is created in CRM or imported from ERP.
- A workflow or flow:
    - Sums line items and compare with header total.
    - Checks for missing VAT or tax values.
- If validation fails:
    - Sets status to “On Hold – Validation Error”.

    - Sends email to finance team with details.
- If validation passes:
    - Starts an approval flow in Power Automate.
    - On approval → marks invoice as Approved and notifies accounting.
    - On rejection → sets status to Rejected with comments.

6. Best Practices and "Loopholes" for Developers

6.1 Workflows

- Always use **field change filters** on update triggers.
- Keep workflows **small, focused**, and use child workflows for shared logic.
- Avoid too many Wait conditions; use scheduled flows for long-running timers.
- Avoid recursion by using flags or version fields.
- Use meaningful names for workflows and include a version or scope in the name (e.g. *Case – Route by Severity v2*).

6.2 Custom Workflow Activities

- Use CWAs for **logic that is hard or impossible in the designer**, not for every small condition.
- Log enough tracing to debug production issues but avoid logging every single detail to reduce noise.
- Do not assume context values; always check that references (lookups) are not null.
- Keep business logic in separate helper classes so it can be reused in plugins as well.

6.3 Power Automate Flows

- Use **solution-aware flows** and **connection references** so you can move flows between Dev, Test, and Prod easily.
- Use **environment variables** for URLs, email addresses, and IDs.
- Group actions into **Scopes** named "Try", "Catch", "Finally" and use **Run After** to implement proper error handling.
- When working with Dataverse rows, always **limit columns** and use pagination to avoid performance issues.
- Name actions and variables clearly, as if someone else will maintain the flow later (because they probably will).